Selección de arquitectura tecnológica inicial

Robot seguidor de personas · Análisis comparativo preliminar (sin inventario de componentes verificado)

1. Planteamiento y restricción clave

El sistema debe equilibrar **rendimiento y compatibilidad** sobre un computador pequeño, evitando sobrecalentamiento o caídas del equipo en un robot de tamaño reducido. Un robot seguidor observa normalmente la **espalda** del usuario, por lo que la identificación no puede apoyarse en el rostro: debe operar sobre el **recorte del cuerpo completo** (forma, textura y color de la vestimenta). El problema se divide en dos etapas: **detección** (¿dónde hay personas?) e **identificación** (¿cuál es el usuario enrolado?).

2. Alternativas de identificación

Alternativa	Funcionamiento	Ventajas	Limitaciones
A. PCA + distancia euclidiana	Proyecta el recorte a un espacio reducido (eigenimages) y compara con la proyección del usuario enrolado.	Muy liviano; entrena rápido; funciona con pocas imágenes de enrolamiento; fácil de explicar.	Sensible a iluminación, pose, escala y fondo. La distancia euclidiana discrimina mal entre personas parecidas.
B. Viola-Jones	Cascada de clasificadores Haar (OpenCV).	Muy rápido en CPU; maduro y disponible.	Es un detector, no un identificador. Solo rostros frontales: inadecuado para un usuario visto de espaldas.
C. PCA + otra distancia	Igual que A con distancia de Mahalanobis (PCA con whitening) o coseno; se puede sumar histograma de color HSV del torso.	Mismo costo que A con mejor discriminación y mayor robustez a la iluminación. Mejora incremental barata.	Limitado ante cambios fuertes de pose o cruces prolongados. Requiere calibrar el umbral de aceptación.
D. Autoencoder + espacio latente	Red convolucional que comprime el recorte a un vector latente; se compara el vector del usuario con el de los candidatos. Se prueban tamaños de imagen (p. ej. 64×128, 96×192).	Aprende rasgos no lineales; tamaño de imagen y de latente ajustables para equilibrar precisión y cómputo.	Requiere dataset y entrenamiento (en PC). El latente optimiza reconstrucción, no identidad: puede requerir pérdida contrastiva/triplet. Mayor riesgo de plazo.

Matriz de decisión ponderada (escala 1 a 5; pesos propuestos, ajustables):

Criterio	Peso	A: PCA-Eucl.	B: Viola-Jones	C: PCA + otra dist.	D: Autoencoder
Robustez de identidad ante cruces	30 %	2	1	3	3
Facilidad de enrolamiento (pocos intentos)	15 %	4	2	4	4
Costo computacional en equipo pequeño	25 %	5	5	5	3
Esfuerzo y riesgo de implementación	15 %	5	4	4	2
Potencial de mejora	15 %	2	1	3	5
Total ponderado (sobre 5)		**3,5**	**2,6**	**3,8**	**3,3**

Lectura: C ofrece la mejor relación entre rendimiento y riesgo para una primera iteración. D tiene mayor techo de mejora pero más riesgo de integración y plazo. B queda descartado como identificador. Los puntajes son conceptuales y deben contrastarse con pruebas.

3. Cómputo: rendimiento vs. compatibilidad

Opción	Rendimiento	Compatibilidad ROS 2	Consumo / temperatura
Raspberry Pi 5 (4 u 8 GB)	Suficiente para detector liviano + PCA en CPU.	Muy buena (Ubuntu + ROS 2).	Requiere refrigeración activa y fuente 5 V / 5 A. Mejor equilibrio costo-comunidad.
Raspberry Pi 4	Justo para detección + identificación en tiempo real.	Muy buena.	Más fría pero más lenta; solo si ya está disponible.
Jetson (Nano / Orin Nano)	Mejor para detector y autoencoder (GPU).	Buena, atada a versiones de JetPack.	Consumo moderado; más potencia, más fricción de instalación.
Mini PC (tipo Intel N100)	Alto, sin GPU dedicada.	Excelente (Ubuntu estándar).	Mayor consumo: compromete la autonomía. Plan B si la Pi no alcanza los FPS.

Medidas para proteger el equipo: disipador con ventilador y monitoreo de temperatura; alimentación de la Pi con **fuente independiente** de la de los motores (las caídas de voltaje al arrancar motores provocan reinicios); y limitar resolución y FPS del procesamiento (p. ej. 320×240 a 10-15 FPS), sin correr la detección en todos los cuadros.

4. Sensores y plataforma de software

Componente	Opción recomendada	Justificación
Cámara	Webcam USB o cámara Pi (RGB).	Suficiente para enrolamiento e identificación. Una cámara de profundidad (RealSense/OAK-D) mejora la distancia, pero sube costo y carga de cómputo.

Componente	Opción recomendada	Justificación
Estimación de distancia	Altura del bounding box calibrada + sensor frontal.	Barato y suficiente para mantener una distancia de seguimiento estable.
Proximidad / obstáculos	Ultrasonido o ToF (VL53L0X).	Aporta seguridad y continuidad de seguimiento complementando a la cámara.
Control de motores	Microcontrolador (Arduino/ESP32) con encoders, enlazado a la Pi.	Separa el control en tiempo real del procesamiento pesado.
Software	ROS 2 + Python + OpenCV + scikit-learn (PCA).	Ecosistema estándar y modular. El autoencoder se exporta a ONNX o TFLite para inferir en la Pi.
Control de seguimiento	PID sobre error lateral (centro del bbox) y error de distancia, publicado como cmd_vel.	Simple, validable y fácil de ajustar.

5. Conclusión: arquitectura inicial recomendada

- **Cómputo:** Raspberry Pi 5 con refrigeración activa y alimentación independiente; mini PC como plan B.
- **Detección:** detector de personas liviano de cuerpo completo (YOLO nano o MobileNet-SSD). Viola-Jones descartado.
- **Identificación (línea base):** opción C, PCA con distancia de Mahalanobis o coseno más histograma de color del torso, sobre el recorte del cuerpo.
- **Evolución experimental:** opción D, autoencoder con pruebas de tamaño de imagen, si C no alcanza las metas. Respaldo adicional: Person Re-ID con embeddings.
- **Sensores y software:** cámara RGB, ultrasonido o ToF, encoders; ROS 2 con microcontrolador para motores y control PID.

Criterio de escalamiento C a D: si en pruebas controladas C no conserva la identidad ante cruces de personas o requiere demasiados intentos de enrolamiento, se avanza a D.

Pendiente: verificar el inventario disponible (modelo de Pi, cámara, sensores, motores) y medir los FPS reales del detector elegido en el hardware definitivo.